Build a Toy Model for Exploring Molecular / Atomic Interactions

What you are doing

You are given the chance to build a toy that shows how atoms might jostle, bounce, and push each other in a small box. This is **not full-scale Molecular Dynamics**, but a playful starting point. You will:

- create arrays for positions and velocities,
- update them using Newton's laws,
- add forces between pairs,
- reflect from walls,
- plot energy and final positions.

Goal. At the end, you will have a small simulator you can tweak — faster steps, stronger springs, more particles — and see what happens.

1 Setting up the world

Install the tools we need (once). If already installed, ignore this step.:

```
pip install numpy matplotlib
```

Now start a new Python file or Jupyter notebook and set up particle positions and velocities:

```
import numpy as np
np.random.seed(0)

N = 12          # number of particles
L = 10.0        # box side length
dt = 0.01       # time step
T  = 1000       # number of steps to simulate

# TODO: create pos, vel, and mass arrays
# Hint: pos should be N x 2 random numbers (x, y) scaled to fit inside box
# vel (vx, vy) should be random velocities
# mass should be all ones
```

TODO

Fill in the code for `pos`, `vel`, and `mass`. Print them once to check shapes: `print(pos.shape, vel.shape)`.

Precision and memory layout. Before moving on, it is worth connecting this array to how C and NumPy actually store numbers in memory. A few useful facts: in C, `sizeof(float)` is 4 bytes and `sizeof(double)` is 8 bytes; a 2D array is really a flattened 1D block of memory, laid out row

by row (row-major order), with consecutive elements sitting at addresses a fixed number of bytes apart; and NumPy exposes this same information through `pos.dtype`, `pos.itemsize`, `pos.nbytes`, and `pos.strides` — you do not need pointers to see it, NumPy will tell you directly.

TODO

1. By default, `np.random.rand()` produces `float64` arrays. Print `pos.dtype`, `pos.itemsize`, `pos.nbytes`, and `pos.strides` for your `(N, 2)` array, and explain each number in one line — the same way you would explain `sizeof(double)` and pointer offsets for an array in C.
2. Make a copy of `pos` stored as `float32`: `pos32 = pos.astype(np.float32)`. Print `pos32.itemsize` and `pos32.nbytes`.
3. If you simulated $N = 100{,}000$ particles instead of 12, how many megabytes would `pos` take as `float64` vs. `float32`? Compute this two ways and check they agree: (i) directly from `pos.itemsize`, and (ii) by hand, using the `sizeof(double)`/`sizeof(float)` values you would measure with C's `sizeof` operator.
4. In one or two sentences, say which precision you would choose for a large simulation, and why — think back to how rounding error (e.g. `0.1 + 0.2`) and summation order can affect a result when you answer.

Hint. `pos.strides` tells you, in bytes, how far apart consecutive rows and columns are in memory — compare this to how you would compute pointer address offsets for an array of doubles in C.

2 Wall reflections (bounce off walls)

Right now, a particle could fly straight out of the box. You need to make it bounce back.

At the end of every step:

1. Look at every particle.
2. If its x-position is < 0, put it back at $x = 0$ and flip its x-velocity (multiply by -1).
3. If its x-position is $> L$, put it back at $x = L$ and flip its x-velocity.
4. Do the same for y-position and y-velocity.

The order matters: **first** detect it crossed the wall, **then** flip its velocity, and **then** clamp its position inside. This prevents particles from getting stuck outside the box or gaining energy by accident.

Here is a clear function you can fill in:

```
def walls_reflect(pos, vel, L):
    # Left wall (x < 0)
    left_hits = pos[:, 0] < 0
    vel[left_hits, 0] *= -1        # flip x velocity
    pos[left_hits, 0] = 0.0        # push back inside

    # Right wall (x > L)
    right_hits = pos[:, 0] > L
    vel[right_hits, 0] *= -1
    pos[right_hits, 0] = L
```

```
    # Bottom wall (y < 0)
    bottom_hits = pos[:, 1] < 0
    vel[bottom_hits, 1] *= -1
    pos[bottom_hits, 1] = 0.0

    # Top wall (y > L)
    top_hits = pos[:, 1] > L
    vel[top_hits, 1] *= -1
    pos[top_hits, 1] = L

    return pos, vel
```

TODO

Test this function: Create a particle just outside the right wall (`pos = [[L+0.1, 5]]`) with a positive x-velocity. Call `walls_reflect`. Check that it ends up at exactly $x = L$ and its velocity becomes negative.

Hint. If you see energy growing after a bounce, check the order: update position first, then reflect. Never flip velocity before you know where the particle is.

3 Finding neighbors: pairwise distances and a cutoff

Before we can compute forces between particles, we need to know how far apart every pair of particles is — and, just as importantly, *which* pairs are close enough to matter. In real molecular simulations, applying a cutoff radius r_c and ignoring pairs farther apart than that is what makes large simulations possible at all: force calculations only need to be done for nearby neighbors, not for every pair in the whole box. The list of who is a neighbor of whom is usually called a **neighbor list**.

For two particles at positions $\mathbf{r}_i = (x_i, y_i)$ and $\mathbf{r}_j = (x_j, y_j)$, the distance between them is

$$r_{ij} = \|\mathbf{r}_i - \mathbf{r}_j\| = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

We want a function that, given all N positions and a cutoff r_c, returns three things: the $N \times N \times 2$ array of pairwise displacement vectors $\mathbf{r}_i - \mathbf{r}_j$, the $N \times N$ matrix of distances r_{ij}, and a boolean $N \times N$ matrix telling us which pairs are neighbors, i.e. $0 < r_{ij} \leq r_c$ (a particle is never its own neighbor). The displacement vectors will come in handy later, when we need the *direction* of a force and not just its size.

```
rc = 2.0   # neighbor cutoff distance

def pairwise_distances(pos, rc):
    N = pos.shape[0]
    # TODO: compute d, the (N, N, 2) matrix of pairwise displacement
    #   vectors r_i - r_j between every pair of particles.
    # Hint: use broadcasting -- pos[:, None, :] - pos[None, :, :]
    #   gives every pairwise displacement vector in one shot, with
    #   no for loop at all.
```

```
    # TODO: compute dist, the (N, N) matrix of distances r_ij
    #   -- the length of each displacement vector in d.

    # TODO: build "neighbors", an (N, N) boolean matrix that is True
    #   wherever 0 < dist <= rc, and False on the diagonal and for
    #   pairs farther apart than rc.

    return d, dist, neighbors
```

TODO

1. Implement `pairwise_distances(pos, rc)` using NumPy broadcasting — no `for` loops. (If you want, first write a slow double-loop version to check your answer against, then delete or comment it out.)
2. Test it on 4 particles you choose by hand. Check that `dist` is symmetric, that its diagonal is exactly zero, and that `neighbors` is `False` on the diagonal.
3. For each particle, count how many neighbors it has: `np.sum(neighbors, axis=1)`. Print this list — you will need it again in Section 4 when you compute forces.
4. Now test it on a larger, randomly generated set of particles (e.g. 200 particles in a bigger box), and report the average number of neighbors per particle for a couple of different cutoff values.

Hint. Try `rc = 2.0` and then `rc = 3.0` on your 4-particle toy example first, and see how the neighbor count changes — this is a quick way to check your mask is doing the right thing before moving on to a larger set of particles.

Keep `d`, `dist`, and `neighbors` around: both the *Pair forces* section and the vectorized "Bonus Pointer" box later reuse `pairwise_distances` instead of recomputing distances from scratch, so make sure it is correct before moving on.

4 Pair forces: springs between particles

You are given this formula:

$$\mathbf{F}_{ij} = -k\,(r - r_0)\,\frac{\mathbf{r}}{\|\mathbf{r}\|}$$

where k is stiffness, r_0 is preferred spacing, r is current distance, and $\mathbf{r}$ is the displacement vector.

```
k  = 15.0
r0 = 0.8
# rc was already defined in Section 3 -- reuse it, do not redefine it here

def pair_forces(pos):
    F = np.zeros_like(pos)
    d, dist, neighbors = pairwise_distances(pos, rc)    # reuse Section 3's function
    # TODO: double loop over i, j (j > i)
    # only apply the spring force if neighbors[i, j] is True
```

```
    # compute the force magnitude using k, r0, and dist[i, j]
    # add equal and opposite forces to F[i], F[j]
    return F
```

TODO

1. Implement the double-loop version of `pair_forces` above, reusing `dist` and `neighbors` from your Section 3 function instead of recomputing distances by hand. Print `F` for a few particles to check it makes sense (zero if the pair is not a neighbor).
2. **C vs. NumPy vs. Python.** Compare how a pure-Python loop, NumPy, and compiled C (`-O0` and `-O2`) perform on this same calculation:
   (a) Write a short C program `pairforce.c` that computes the same double-loop spring force for a set of particles (read in or hardcode N, k, r_0, r_c, and random positions), compiled with `gcc -O2 -lm`.
   (b) Time your Python double-loop `pair_forces`, your C version, and the vectorized version from the "Bonus Pointer" box below, for at least four values of N (e.g. 50, 200, 500, 1000).
   (c) Make one wall-time vs. N plot (log scale on the time axis) with all three curves on it, comparing the loop-based and vectorized approaches directly.
3. In two or three sentences, explain the ordering you observe — which is fastest, which is slowest, and why? Refer to what you know about compiled vs. interpreted execution and about vectorization overhead.

5 Updating positions and velocities

Now you will make the particles move step by step. At every small time step Δt, do three things in this order:

1. Find the total force $\mathbf{F}$ on each particle (use your `pair_forces` function).
2. Change velocity using: $\mathbf{v} = \mathbf{v} + (\mathbf{F}/m)\,\Delta t$
3. Change position using: $\mathbf{x} = \mathbf{x} + \mathbf{v}\,\Delta t$
4. Make sure no particle is outside the box. If it is, flip its velocity (your wall bounce function).

In code, that looks like:

```
def step_once(pos, vel, mass, dt, L):
    # 1. Compute forces
    F = pair_forces(pos)

    # 2. Update velocities using F = m a
    acc = F / mass[:, None]
    vel = vel + acc * dt

    # 3. Update positions using new velocities
    pos = pos + vel * dt

    # 4. Bounce from walls
    walls_reflect(pos, vel, L)
```

```
    return pos, vel
```

TODO

Fill in any missing parts in this function. Then run it in a loop for 50–100 steps and print the first particle's position each time. Do you see it move smoothly inside the box?

6 Energy check

Define a function to measure kinetic energy:

$$E_k = \frac{1}{2} \sum mv^2$$

```
def kinetic_energy(vel, mass):
    # TODO: compute and return total KE (use np.sum)
```

Collect KE in a list during the simulation and plot later.

7 A smoother update recipe

The simple recipe you just coded works, but if you look at your kinetic energy plot you might notice that it slowly drifts up or down. Here is a slightly improved way to update positions and velocities that usually keeps energy more steady.

At every small time step Δt, do this:

1. Compute the force $\mathbf{F}$ on each particle.
2. Use half of this force to change the velocity halfway:

$$\mathbf{v}_{\text{half}} = \mathbf{v} + \frac{1}{2} \frac{\mathbf{F}}{m} \Delta t$$

3. Move the particles using this half-updated velocity:

$$\mathbf{x}_{\text{new}} = \mathbf{x} + \mathbf{v}_{\text{half}} \Delta t$$

4. Compute the new forces at these new positions.
5. Use half of these new forces to finish the velocity update:

$$\mathbf{v}_{\text{new}} = \mathbf{v}_{\text{half}} + \frac{1}{2} \frac{\mathbf{F}_{\text{new}}}{m} \Delta t$$

6. Bounce from walls if needed.

In code, this might look like:

```
def step_smooth(pos, vel, mass, dt, L, F_prev=None):
    # 1. Force at old positions
```

```
    if F_prev is None:
        F_prev = pair_forces(pos)
    a_prev = F_prev / mass[:, None]

    # 2. Half velocity update
    vel_half = vel + 0.5 * a_prev * dt

    # 3. Move positions
    pos_new = pos + vel_half * dt
    walls_reflect(pos_new, vel_half, L)

    # 4. New forces
    F_new = pair_forces(pos_new)
    a_new = F_new / mass[:, None]

    # 5. Finish velocity update
    vel_new = vel_half + 0.5 * a_new * dt

    return pos_new, vel_new, F_new
```

TODO

Replace your previous update function with this new one and re-run the simulation. Plot kinetic energy again. Is it steadier than before? Try with a bigger `dt` — how does the energy behave?

TODO

Explain in one line what you observe in the KE plot. Is energy roughly constant?

Bonus Pointer: Vectorized Forces

Once your double loop works, you can compute all the forces at once using NumPy broadcasting for speed — and you have already built the hard part of this in Section 3. Your `pairwise_distances(pos, rc)` function already returns the displacement vectors `d`, the distances `dist`, and the neighbor mask `neighbors`, so `pair_forces` can simply reuse it instead of recomputing anything:

```
d, dist, neighbors = pairwise_distances(pos, rc)  # reuse Section 3's function

u = d / (dist[:, :, None] + 1e-12)        # unit vectors
fmag = -k * (dist - r0) * neighbors       # force magnitude, zero outside the cutoff
F = np.sum(fmag[:, :, None] * u, axis=1)  # sum forces per particle
```

This replaces the loop with array math. It looks fancy, but it's just the same steps done in parallel, built directly on top of the function you already wrote and tested in Section 3.

Checklist

- ☐ Particles stay inside the box.

- ☐ Forces are equal and opposite (check by summing F).
- ☐ Kinetic Energy (KE) stays roughly constant with the smoother update given in step 6.
- ☐ Smaller `dt` makes result more stable.
- ☐ `pos.dtype`, `itemsize`, `nbytes`, and `strides` are printed and explained, and the float32-vs-float64 memory estimate for $N = 100{,}000$ matches the by-hand calculation using C's `sizeof`.
- ☐ `pairwise_distances(pos, rc)` returns `d`, `dist`, and `neighbors` using broadcasting (no loops), and is tested on a hand-built example and on a larger set of randomly placed particles.
- ☐ Both `pair_forces` (loop version) and the vectorized Bonus Pointer version reuse `pairwise_distances` rather than recomputing distances.
- ☐ A C version of `pair_forces` (`pairforce.c`) exists, and a wall-time vs. N plot compares Python-loop, C, and NumPy-vectorized versions, with a short written explanation of the ordering observed.